#3 Some bitwise algebra to return the value: first retrieve the buffer element, then puts it in AND with a mask that extracts a single bit (at the right position), finally it shifts the extracted value back so that the result will either be a 0 or a 1. We could save a left shift by preparing a constant array of `BITS_PER_INT` masks, and use bit as an index to decide which mask should be applied.

Listing 4.6, instead, shows the writing counterpart, method `writeBit`; you might be surprised to see that we don't pass the value to write: but since (this version of) Bloom filter doesn't support deleting elements, we can only write a 1, we never write zeros.

Listing 4.6 Method writeBit

```
function writeBit(bitsArray,
index)                                          #1
  (element, bit) ← findBitCoordinates(bitsArray,
index)                        #2
  bitsArray[element] ← bitsArray[element] | (1 <<
bit)                         #3
  return bitsArray
```

#1 Method `writeBit` takes the bits arrays and the index of the bit where a 1 should be written; it returns the bits array after modifying it.
#2 Retrieve the element index and offset for the bit in the bits array.
#3 Some more bitwise algebra to store the value. Put current buffer’s byte in OR with a mask having a single 1 in the position where we need to write, and then store the result